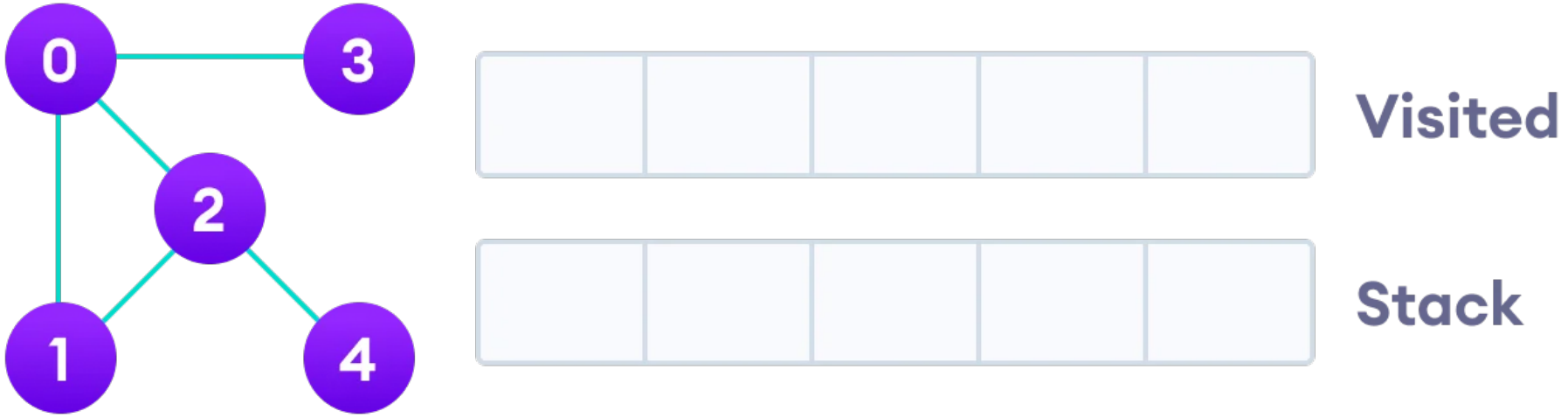
Unidad 7

CLASE 7.3 – DIJKSTRA

DFS (profundidad)



```python
# Tomado de: https://www.programiz.com/dsa/graph-dfs
# DFS algorithm in Python


# DFS algorithm
def dfs(graph, start, visited=None):
    if visited is None:
        visited = set()
    visited.add(start)

    print(start)

    for next in graph[start] - visited:
        dfs(graph, next, visited)
    return visited


graph = {'0': set(['1', '2']),
         '1': set(['0', '3', '4']),
         '2': set(['0']),
         '3': set(['1']),
         '4': set(['2', '3'])}

dfs(graph, '0')
```

El algoritmo anterior presenta un problema

¿Cuál es?

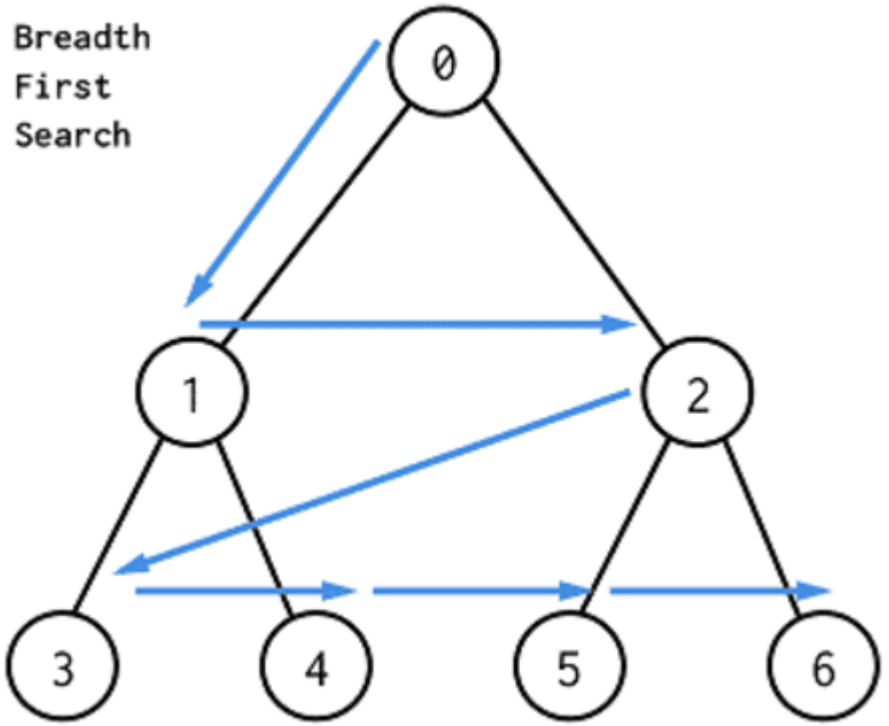
Pista: tiene que ver con el modo de funcionar del **for**

¿Cómo puede solucionarse?

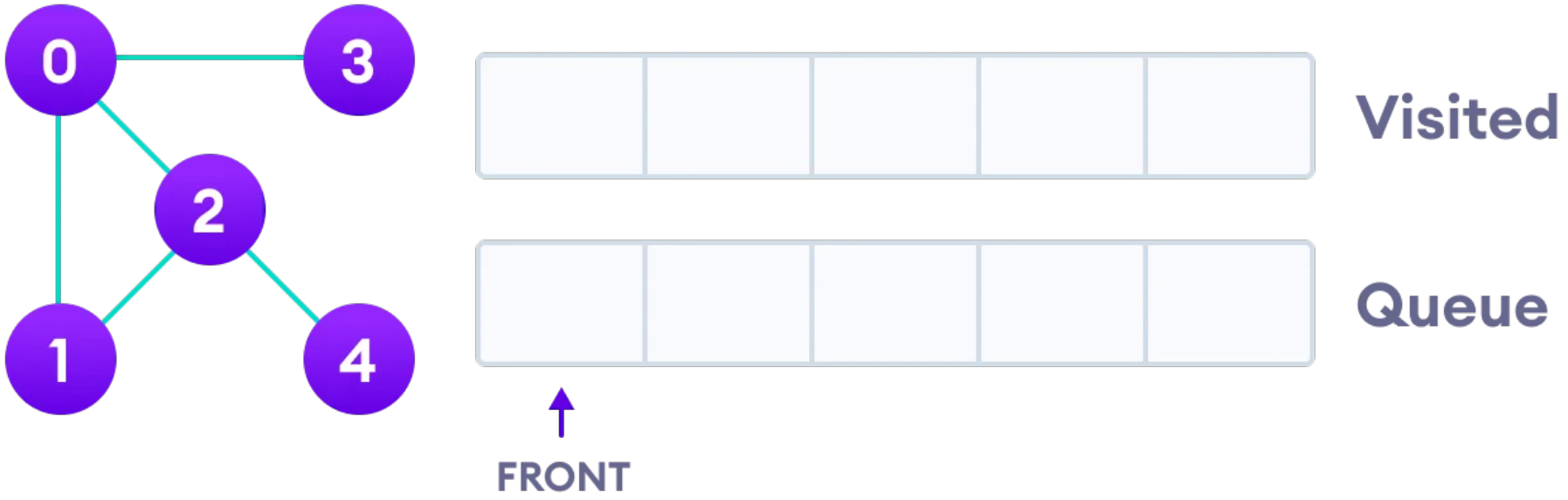
Con un **while**

Con un **if**

Recorrido BFS



BFS (anchura)



```python
# BFS que aparece en Smith

from collections import deque


def bfs(graph, start):

    visited, queue = set(), deque([start])
    visited.add(start)

    while queue:
        vertex = queue.popleft()
        print(vertex)

        for neighbour in graph[vertex]:
            if neighbour not in visited:
                visited.add(neighbour)
                queue.append(neighbour)
```

```python
# Dijkstra que aparece en Smith
import heapq


def dijkstra(graph, start):
    D = {v: float('infinity') for v in graph}
    D[start] = 0
    pqueue = [(0, start)]


    while pqueue:
        (current_distance, vertex) = heapq.heappop(pqueue)


        for neighbor, w in graph[vertex].items():
            distance = current_distance + w

            if distance < D[neighbor]:
                D[neighbor] = distance
                heapq.heappush(pqueue, (distance, neighbor))

    return D
```